Dynamic Programming

Optimal Substructure, Overlapping Subproblems, and Classical Applications

Asaduzzaman Herok

June 15, 2026

Lecture Roadmap

Part I — Foundations

- What is Dynamic Programming?
- Optimal Substructure
- Overlapping Subproblems
- Memoization (Top-Down)
- Tabulation (Bottom-Up)
- Space Optimisation
- Fibonacci — warm-up

Part II — Applications

- Coin Row / Coin Change
- 0-1 Knapsack + Memory Functions
- Optimal Binary Search Trees
- Warshall's Algorithm (Transitive Closure)
- Floyd's Algorithm (All-Pairs Shortest Paths)
- Complexity Summary Table

Reading

CLRS 4e — Chapter 14; Levitin — Chapter 8

Dynamic Programming — Core Concepts

Part I: Foundations

What is Dynamic Programming? I

Dynamic Programming (DP)

An algorithm design paradigm that solves complex problems by breaking them into **overlapping subproblems**, solving each subproblem **exactly once**, and **storing** the result for reuse.

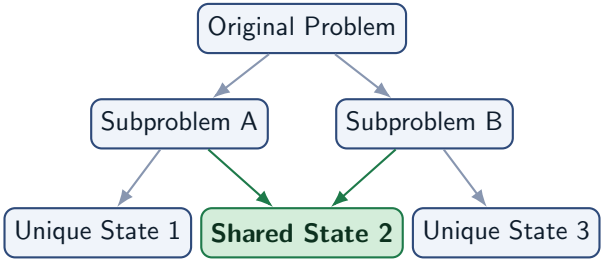
History:

- Invented by **Richard Bellman** (1950s) for multistage decision optimization.
- "Programming" = *planning* (not coding).
- Now a general-purpose technique in CS.

Compared to Divide & Conquer:

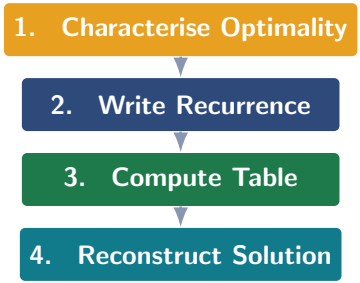
- D&C: subproblems are **disjoint**
- DP: subproblems **overlap** — reuse is the key saving

What is Dynamic Programming? II



The Four-Step DP Method (CLRS)

1. **Characterise** the structure of an optimal solution.
   — What does optimality look like?
2. **Recursively define** the value of an optimal solution.
   — Write the recurrence $\mathrm{OPT}(n) = f(\mathrm{OPT}(\text{smaller}))$
3. **Compute** the optimal value, typically *bottom-up*.
   — Fill a table, smallest subproblems first
4. **Construct** an optimal solution from the table.
   — Backtrack through the table (if needed)



Key Idea

Steps 1–3 give the **value**; step 4 gives the **solution**.
Skip step 4 if only the value is needed.

Optimal Substructure I

Optimal Substructure Property

A problem exhibits **optimal substructure** if an optimal solution to the problem contains optimal solutions to its subproblems. This is *Bellman's Principle of Optimality*.

Formally: If S^* is an optimal solution for problem P, and S^* is constructed from solutions $s_1, s_2, \ldots$ to sub-problems $P_1, P_2, \ldots$, then each s_i must be optimal for P_i.

Proof technique — "cut & paste": Suppose s_i is not optimal. Replace it with s_i^* (the optimal). The resulting solution is strictly better — **contradiction**.

Optimal Substructure II

Holds for:

- Shortest paths (Bellman-Ford, Floyd)
- 0-1 Knapsack
- Optimal BST
- Coin change, Fibonacci

Fails for:

- Longest *simple* path in a graph (subpaths may share vertices)
- Unweighted longest common subsequence with constraints

Key Idea

Always verify optimality holds before applying DP!

Overlapping Subproblems I

Overlapping Subproblems

A problem has **overlapping subproblems** when the recursive algorithm visits the *same* subproblem multiple times. DP exploits this by computing each subproblem **only once**.

Why does this matter?

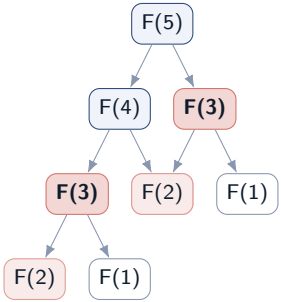
- Naive recursion: exponential time (recomputes!)
- DP (with memoisation): polynomial time

Contrast with D&C:

- Merge sort / quicksort: subproblems *disjoint*
- No benefit from storing results
- DP: subproblems *share* sub-subproblems

Overlapping Subproblems II

Fibonacci call tree for $F(5)$:



F(3) computed **twice**, F(2) **three times**!

Two DP Strategies: Memoisation vs. Tabulation I

Top-Down: Memoisation

- Write the natural recursion
- Add a **cache** (hash map / array) initialised to `null`
- Before computing, **check cache**
- After computing, **store result**
- Computes *only needed* subproblems

+ Intuitive, mirrors recursion
+ Lazy: skips unnecessary states
– Call-stack overhead (recursion)
– Harder to analyse precisely

Bottom-Up: Tabulation

- Identify ordering of subproblems
- Fill a **table** from smallest to largest
- No recursion — pure iteration
- **All** subproblems computed
- Straightforward space optimisation

+ No recursion overhead
+ Easy space reduction
+ Cache-friendly access pattern
– Must solve all subproblems

Two DP Strategies: Memoisation vs. Tabulation II

Key Distinction

Both achieve the same **asymptotic time complexity**. Tabulation is generally preferred for **space optimisation**. Memoisation is useful when the subproblem space is **sparse**.

Memoization: Code Pattern I

Without memoization (exponential):

Naive Fibonacci

```
fib(n):
    if n ≤ 1:  return n
    return fib(n-1) + fib(n-2)

// Time:  O(2^n) | recomputes
same calls
```

With memoization (linear):

Memoised Fibonacci

```
memo = {0:  0, 1:  1}

fib(n):
    if n in memo:
        return memo[n]
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]

// Time:  O(n), Space:  O(n)
```

Memoization: Code Pattern II

General memoization

Template

```
// initialise table T to -1 (null)
T[0..n] ← −1

solve(i):
    if T[i] ≥ 0:
        return T[i] // cached!
    T[i] ← ...compute...
    return T[i]
```

When to Use Memoisation

- Subproblem space is large but **sparse**
- Natural recursive formulation is clear
- Used in: Knapsack (memory functions), OBST

Tabulation: Code Pattern I

Tabulation (bottom-up):

Tabulated Fibonacci

```
fib_tab(n):
    F[0] ← 0; F[1] ← 1
    for i ← 2 to n:
        F[i] ← F[i-1] + F[i-2]
    return F[n]
// Time:  O(n), Space:  O(n)
```

Execution trace ($n = 6$):

i	0	1	2	3	4	5	6
$F[i]$	0	1	1	2	3	5	**8**

Tabulation: Code Pattern II

General tabulation

Template

```
// 1. Set base cases
F[0..base] ← known values

// 2. Fill table in dependency order
for i ← 1 to n:
    F[i] ← recurrence(F[...])

// 3. Answer is in F[n]
return F[n]
```

Key Insight

- The table fills in a topological order of subproblem dependencies.
- Every entry you compute depends only on **already-filled** entries.

Space Complexity Optimization I

Observation: Many DP recurrences only need a *small window* of previous values, not the full table.

General principle:

- If $F[i]$ depends only on $F[i-1]$ and $F[i-2]$: $\Rightarrow \mathcal{O}(1)$ space
- If $F[i][j]$ depends only on row $i-1$: $\Rightarrow$ use two rows: $\mathcal{O}(n)$ instead of $\mathcal{O}(n^2)$
- Coin change (1D): $\mathcal{O}(n)$ trivially
- Knapsack: From $\mathcal{O}(nW)$ to $\mathcal{O}(W)$ (with careful traversal order — see later)
- OBST, Floyd: harder; often need $\mathcal{O}(n^2)$

Space-Optimized Fibonacci

```
fib_opt(n):
    prev2 ← 0
    prev1 ← 1
    for i ← 2 to n:
        curr ← prev1 + prev2
        prev2 ← prev1
        prev1 ← curr
    return prev1
// Time:  O(n), Space:  O(1)
```

Idea: Only keep last two values.

Space Complexity Optimization II

Trade-off Warning

Space-optimised DP often **loses the ability to backtrack** and reconstruct the solution. Keep full table if solution reconstruction is required.

Classical DP Problems — Deep Dives

Part II: Applications

Coin Row Problem I — Selecting non-adjacent coins for maximum value

Problem

Given n coins in a row with values $c_1, c_2, \ldots, c_n$, pick up the maximum total value subject to: **no two adjacent coins may be picked**.

Optimal Substructure

Consider coin c_n (last):

- **Include** c_n: add c_n + best from coins $1 \ldots n-2$
- **Exclude** c_n: best from coins $1 \ldots n-1$

$$F(n) = \max\{c_n + F(n-2),\ F(n-1)\}\ F(0) = 0, \quad F(1) = c_1$$

Coin Row Problem II — Selecting non-adjacent coins for maximum value

CoinRow(C[1..n])

```
F[0] ← 0; F[1] ← C[1]
for i ← 2 to n do
    F[i] ← max(C[i] + F[i-2], F[i-1])
return F[n]
```

Backtrack: Optimal $= \{c_1, c_4, c_6\} = \{5, 10, 2\}$

Complexity

Time: $\Theta(n)$
Space: $\Theta(n)$ (or $\mathcal{O}(1)$ with just 2 vars)

Example: Coins $= [5, 1, 2, 10, 6, 2]$

i	0	1	2	3	4	5	6
$C[i]$	—	5	1	2	10	6	2
$F[i]$	0	5	5	7	15	15	**17**

Calculation:

$$F[2] = \max(1 + 0, 5) = 5$$
$$F[3] = \max(2 + 5, 5) = 7$$
$$F[4] = \max(10 + 5, 7) = 15$$
$$F[5] = \max(6 + 7, 15) = 15$$
$$F[6] = \max(2 + 15, 15) = \mathbf{17}$$

Problem

Given denominations $d_1 < d_2 < \cdots < d_m$ (with $d_1 = 1$) and amount n, find the **minimum number of coins** that sum to n.

Optimal Substructure:

The last coin used has denomination d_j. After using it, we need minimum coins for $n - d_j$.

- Try every valid denomination $d_j \leq n$
- Pick the one minimising $F(n - d_j) + 1$

$$F(n) = \min_{j:\, n \geq d_j} \{F(n - d_j)\} + 1$$
$$F(0) = 0$$

Coin Change Problem II — Minimum coins to make change for amount n

ChangeMaking(D[1..m], n)

```
F[0] ← 0
for i ← 1 to n do
    temp ← ∞; j ← 1
    while j ≤ m and i ≥ D[j] do
       temp ← min(F[i - D[j]], temp)
       j ← j + 1
   F[i] ← temp + 1
return F[n]
```

Example: $n = 6$, denominations $= \{1, 3, 4\}$

n	0	1	2	3	4	5	6
$F[n]$	0	1	2	1	1	2	**2**

Key computations:

$$F[1] = F[0] + 1 = 1 \quad (d = 1)$$
$$F[3] = \min(F[2], F[0]) + 1 = 1 \quad (d = 3)$$
$$F[4] = \min(F[3], F[1], F[0]) + 1 = 1 \quad (d = 4)$$
$$F[6] = \min(F[5], F[3], F[2]) + 1 = 2 \quad (d = 3)$$

Solution: Two coins of denomination 3
Backtrack: by recording which d_j achieved minimum.

Complexity

Time: $\mathcal{O}(nm)$ Space: $\Theta(n)$

Coin Change: Important Variants I

Variant 1 — Minimum Coins (done)

- Recurrence: $F(n) = \min_{d_j \le n}\{F(n-d_j)\} + 1$
- Unlimited coin supply
- Time: $\mathcal{O}(nm)$, Space: $\mathcal{O}(n)$

Variant 2 — Count of Ways

- How many ways to make change for n?
- ways$(n) = \sum_{d_j \le n}$ ways$(n - d_j)$
- Same structure, addition instead of min

Variant 3 — Limited Supply

- At most k_j coins of denomination d_j
- Becomes a bounded knapsack variant
- Extra dimension for coin count

Variant 4 — Coin Row (no adjacency)

- As seen: $F(n) = \max(c_n + F(n-2), F(n-1))$
- Fixed arrangement, each coin used at most once

Coin Change: Important Variants II

Greedy vs. DP:

Key Idea

Greedy ("always use largest coin") works for standard US/EU denominations, but **fails** in general. Example:
Denominations: $\{1, 3, 4\}$, Amount $= 6$
Greedy: $4 + 1 + 1 = 3$ coins
DP: $3 + 3 = 2$ coins ✓
DP is the correct general approach.

0-1 Knapsack

Given n items with weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$, and a knapsack of capacity W: select a subset of items to **maximise total value** subject to **total weight** $\leq W$. Each item is either included (1) or not (0).

Optimal Substructure

Let $F(i, j) =$ max value from items $1 \ldots i$ with capacity j.

Two cases for item i:

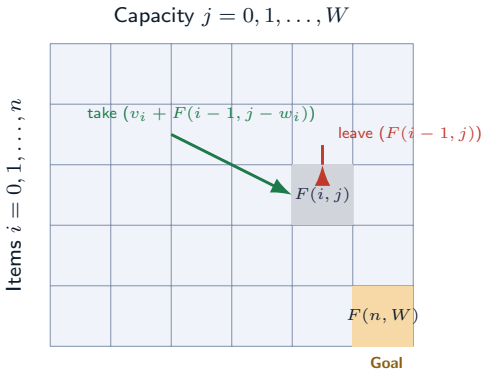
- **Leave item** i: $F(i-1, j)$
- **Take item** i (if $j \geq w_i$): $v_i + F(i-1, j-w_i)$

0-1 Knapsack Problem II — Formulation and Recurrence

$$F(i,j) = \begin{cases} \max\{F(i-1,j),\ v_i + F(i-1, j-w_i)\} & j \geq w_i \\ F(i-1,j) & j < w_i \end{cases}$$

$$F(0,j) = 0\ \forall j; \quad F(i,0) = 0\ \forall i$$

State space interpretation:



Fill order: Row by row (or column by column). Each cell depends on the row **above** it.

Complexity

Time: $\Theta(nW)$ Space: $\Theta(nW)$

Note: *pseudo-polynomial* — W is the numeric value, not bit-length

Items:

Item	1	2	3	4
Weight	2	1	3	2
Value	\$12	\$10	\$20	\$15

DP Table $F(i, j)$:

$i \setminus j$	0	1	2	3	4	5
0 (base)	0	0	0	0	0	0
1 (w=2, v=12)	0	0	12	12	12	12
2 (w=1, v=10)	0	10	12	22	22	22
3 (w=3, v=20)	0	10	12	22	30	32
4 (w=2, v=15)	0	10	15	25	30	**37**

Optimal value: $F(4, 5) = \$37$

Backtracking to find composition:

- $F(4, 5) > F(3, 5) \Rightarrow$ include item 4 ($w = 2$); go to $F(3, 3)$
- $F(3, 3) = F(2, 3) \Rightarrow$ skip item 3
- $F(2, 3) > F(1, 3) \Rightarrow$ include item 2 ($w = 1$); go to $F(1, 2)$
- $F(1, 2) > F(0, 2) \Rightarrow$ include item 1 ($w = 2$)

$\Rightarrow$ **Optimal set:** $\{$**item 1, item 2, item 4**$\}$

Memory Functions: Top-Down DP for Knapsack I

Motivation

Bottom-up computes **all** $n \times W$ cells. For some instances, only a **small subset** of cells is needed. Memory functions (memoised top-down) compute **only necessary** subproblems.

Memory Functions: Top-Down DP for Knapsack II

Algorithm:

MFKnapsack(i, j)

```
// F init:  row 0 and col 0 = 0, rest = -1
   if F[i, j] ≥ 0:
       return F[i, j] // cached
   if j < Weights[i]:
       val ← MFKnapsack(i-1, j)
   else:
       val ← max(MFKnapsack(i-1, j), Values[i] + MFKnapsack(i-1, - Weights[i]))
   F[i, j] ← val
return F[i, j]
```

Memory Functions: Top-Down DP for Knapsack III

Computed cells for the worked example:

$i \backslash j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	—	12	22	—	22
3	0	—	—	22	—	32
4	0	—	—	—	—	**37**

"—" = never computed; blue = computed.

Analysis

Only **11 of 20** non-trivial cells computed. Efficiency class: same $\mathcal{O}(nW)$ worst case, but can be significantly faster in practice.

Knapsack: Space Optimisation to $\mathcal{O}(W)$ I

Key observation: $F(i, j)$ depends only on **row** $i - 1$ of the table. $\Rightarrow$ We only need **one array of size** $W + 1$, updated in-place.

Critical: Must iterate j from **right to left** (W down to w_i) to avoid using the same item twice.

Space-Optimised Knapsack

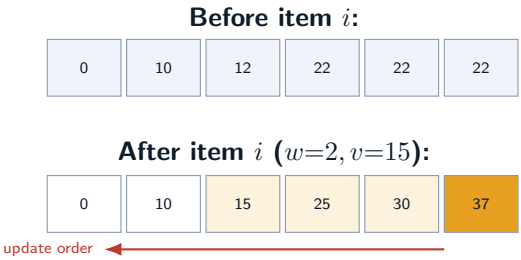
```
F[0..W] ← 0 // single array
for i ← 1 to n do
    for j ← W downto w[i] do
      F[j] ← max(F[j], v[i] + F[j - w[i]])
return F[W]
```

Knapsack: Space Optimisation to $\mathcal{O}(W)$ II

Complexity

Time: $\Theta(nW)$ Space: $\Theta(W)$

Why right-to-left?



Knapsack: Space Optimisation to $\mathcal{O}(W)$ III

Warning

Iterating **left-to-right** would allow an item to be selected **multiple times** — this solves the unbounded knapsack instead!

Trade-off: Cannot reconstruct optimal subset directly; need additional bookkeeping if the subset is required.

Optimal Binary Search Tree — Motivation I

Problem

Given n keys $a_1 < a_2 < \cdots < a_n$ with search probabilities $p_1, p_2, \ldots, p_n$ ($\sum p_i = 1$), build the BST that **minimises the expected number of comparisons** in a successful search.

Expected comparisons in tree T:

$$E[T] = \sum_{i=1}^{n} p_i \cdot \text{depth}(a_i)$$

where depth of root $= 1$.

Optimal Binary Search Tree — Motivation II

Why not just build any BST?

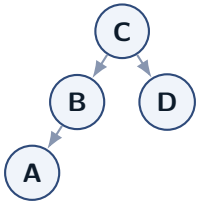
- A balanced BST minimises *worst-case* depth
- OBST minimises *expected* comparisons
- High-probability keys should be near root

Key Idea

Total BSTs with n keys = Catalan number $C_n \sim \frac{4^n}{n^{1.5}}$
Exhaustive search is completely infeasible.

Example: Keys A, B, C, D with probs 0.1, 0.2, 0.4, 0.3

Optimal Binary Search Tree — Motivation III



$$E = 0.1{\cdot}3 + 0.2{\cdot}2 + 0.4{\cdot}1 + 0.3{\cdot}2 = \mathbf{1.7}$$

This is the **optimal** BST for this example.
Key C (highest probability 0.4) is at the root.

Optimal BST — Recurrence and Algorithm I

Let $C(i, j) =$ min expected comparisons in optimal BST for keys $a_i, \ldots, a_j$.

Choose root a_k ($i \le k \le j$):

- Left subtree: optimal BST for $a_i, \ldots, a_{k-1}$
- Right subtree: optimal BST for $a_{k+1}, \ldots, a_j$
- When entire subtree becomes a child, depths $+1$

$$C(i,j) = \min_{i \le k \le j} \{C(i,k-1) + C(k+1,j)\} + \sum_{s=i}^{j} p_s$$

$C(i, i-1) = 0$ (empty tree); $\quad C(i,i) = p_i$

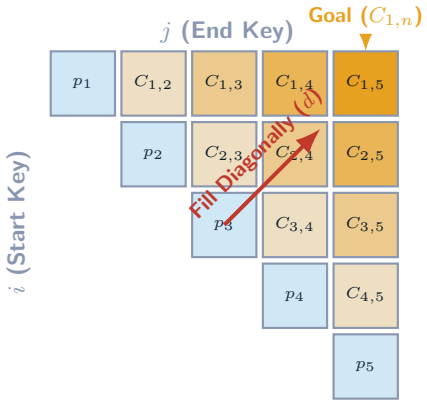
Optimal BST — Recurrence and Algorithm II

OptimalBST(P[1..n]) — sketch

```
for d ← 1 to n-1 do // diagonal
   for i ← 1 to n-d do
      j ← i + d
      for k ← i to j do
         find min C[i,k-1] + C[k+1,j]
      C[i,j] ← minval + sum(P[i..j])
return C[1,n], R
```

Table filling order (diagonal by diagonal):

Optimal BST — Recurrence and Algorithm III



Result table for A,B,C,D:

Optimal BST — Recurrence and Algorithm IV

$C(i,j)$	j=1	j=2	j=3	j=4
i=1	0.1	0.4	1.1	**1.7**
i=2	—	0.2	0.8	1.4
i=3	—	—	0.4	1.0
i=4	—	—	—	0.3

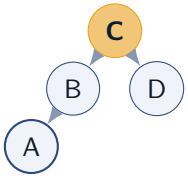
Complexity

Time: $\mathcal{O}(n^3)$ (naive) or $\mathcal{O}(n^2)$ (Knuth) Space: $\mathcal{O}(n^2)$

Optimal BST — Solution Reconstruction I

Root Table $R(i,j)$: Record which k achieved the minimum in $C(i,j)$.

$R(i,j)$	j=1	j=2	j=3	j=4
i=1	1	2	3	**3**
i=2	—	2	3	3
i=3	—	—	3	3
i=4	—	—	—	4



Reconstruction procedure:

1. $R(1,4) = 3$: root is key **C**
2. Left subtree: $R(1,2) = 2$: root is **B**
3. $R(1,1) = 1$: left child of B is **A**
4. Right subtree: $R(4,4) = 4$: root is **D**

Verification:

$$E = 0.1 \cdot 3 + 0.2 \cdot 2 + 0.4 \cdot 1 + 0.3 \cdot 2$$
$$= 0.3 + 0.4 + 0.4 + 0.6 = \mathbf{1.7}$$

Compare to balanced tree E = 2.1 — OBST is better!

Optimal BST — Solution Reconstruction II

Key Idea

Knuth's improvement: Root table entries are monotone along rows/cols, restricting k search to $[R(i, j-1), R(i+1, j)]$ — achieves $\mathcal{O}(n^2)$.